Project #3:
Recovering the Lost Cats!

1 The Story

It is the distant future—the year is 2035. You are one of the lucky survivors of the GPT war of the 2030s struggling to piece society back together. You live in the township of Grunvul, the capital of the newly reorganized state of Outer Karolynia.

Your team is searching for lost technologies to use in your new society, and has uncovered a stockpile of old magnetic hard drives. You suspect that they may contain data that is useful for (1) getting important infrastructure back up and running, (2) recovering lost technologies, and (3) entertaining a depressed population. Specifically, you have heard old timers talk about the old days when magnetic disks were full of cat images. None of you have ever seen a cat (they were VibeNet's first target), but they say that people used to spend hours looking at them before the war, and you and your team are mildly obsessed with the idea of seeing one. Who knows, the disks might even contain cat genome data that could lead to their recovery from extinction! It's up to you to recover the feline memories and dreams of the human race.

The disks are formatted using the FAT16 file system. Your team is tasked with creating CAT16[1], a library that can read from and write to FAT16 disks — well for simplicity we will work with disk images, so I don't have to distribute physical hard drives.

2 Your Mission: CAT16

In this project, you will create a library, called **cat16.a**, that parses FAT16-formatted disk images, and provides functions for reading and writing to those disk images. There is extra credit available if you are able to access files that someone has tried to delete. Your library will implement the API

[1] Cat Acquisition Tool, 16-bit edition!

Listing 1: cat16.h — your library's interface.

```
#ifndef CAT16_H
#define CAT16_H

//additional details in the repository

cat16_volume_t *cat16_mount(const char *image_path);
int cat16_unmount(cat16_volume_t *v);

int cat16_writefile(cat16_volume_t *v, const char *path,
    uint8_t *filedata, size_t bytes_to_write);
uint8_t *cat16_readfile(cat16_volume_t *, const char *
    path, size_t *bytes_read);
cat16_dirent_t *cat16_listdir(cat16_volume_t *v, const
    char *path, size_t *entries_read);
int cat16_stat(cat16_volume_t *v, const char *path,
               cat16_stat_t *out_stat);

int cat16_mkdir(cat16_volume_t *v, const char *path);
int cat16_unlink(cat16_volume_t *v, const char *path);
int cat16_rmdir(cat16_volume_t *v, const char *path);

const char *cat16_vstrerror(cat16_volume_t *v);
const char *cat16_strerror(int error_code);

#endif
```

shown in Listing **??**. This API will be defined in a file, called `cat16.h`, which I will put in the class git repository. **You should use the header file as is. Do not change it, or we may not be able to test your project submission.**

The header file in the repository describes in detail how each of these functions should behave. Your library should not produce any output other than the output described in the header file. Any debug output is likely to confuse the autograder.

3 About FAT16

The FAT file system is a fairly simple file system that was used on nearly all disks during the DOS and Windows 9x era. It is still commonly used on flash drives and supported by nearly all operating systems. A File Allocation Table (FAT) is a data structure used on a disk (hard disk, flash drive, or

Listing 2: Organization of the FAT16 file system.

Section	Boot Sector	FAT1	FAT2	Root Directory	Data
Sector(s)	0	1–32	33–64	65–96	97–8195
Cluster(s)	—	—	—	—	2–8100

floppy disk) to store that status and location of the data "clusters" that are stored on the disk. Without the FAT, which acts as a table of contents, the disk is unreadable. There are several versions of the FAT file system (*e.g.,* FAT12, FAT16, FAT32). The number refers to the size of the FAT entries (12, 16, or 32 bits). FAT16 will be the focus of this assignment.

The organization of a FAT16-formatted disk image that we will use in this project is shown in Listing 2. The storage space is divided up into **sectors**. The first sector (the boot sector) contains various information about the disk. The next 64 sectors contain two copies of the FAT (there are 2 copies, in case one gets corrupted). The next 32 sectors contain the root directory. Note that the root directory has a fixed size, which limits the number of entries that it can contain. Finally, the remaining sectors contain the actual file data, or data **clusters**. The first segment after the root directory, is cluster `0x002` (`0x000` is used to indicate an empty cluster, and `0x001` isn't used).

3.1 The Boot Sector

The boot sector (the first sector on the disk image) describes the geometry of the file system. It's formatting is described in the following table.

Offset	Size	Field	Typically
0	3	Jump Instruction	`EB 3C 90`
3	8	OEM Name	`mkfs.fat`
11	2	Bytes per sector	`0x200 (512)`
13	1	Sectors per cluster	1
14	2	Reserved sectors	1 (boot sector)
16	1	Number of FATs	2
17	2	Number of Root Entries	`0x200 (512)`
19	2	Total Sectors (16-bit)	`0x2000 (8192)`
21	1	Media descriptor	`0xF8`
22	2	Sectors per FAT	`0x0020 (32)`
24	2	Sectors per track	`0x0020 (32)`
26	2	Number of heads	2
28	4	Hidden sectors	0
32	4	Total sectors (32-bit)	0
36	1	Drive number	
37	1	Reserved	
38	1	Extended boot signature	
39	4	Volume ID	
43	11	Volume label	
54	8	Filesystem type	FAT16
62	—	Boot code / paadding	
510	2	Boot signature	`55 AA`

While this allows for a lot of different variations, for simplicity your library is only required to read from and write to disk images with the following properties: Image size = 4MB (or `0x4L « 20L`), sector size = 512, sectors per cluster = 1, sectors per fat = 32, number of FATs = 2, number of root directory entries = 512, total sectors = 0x2000. If I try to mount a disk image with different geometry, your library should just fail to mount it (`cat16_mount` should return NULL).

3.2 The File Allocation Table

The File Allocation Table contains one entry for every cluster in the data area. In FAT16, each entry is 16-bits long.

Entries: One tricky thing about FAT is that all integers are stored in little endian format. This occasionally causes confusion. Just remember if you see two bytes (`02 00`) that are supposed to be an integer, the value is `0x0002`.

The first two entries: The first cluster in the data area is cluster #2, leaving the first two entries of the FAT unused. The first entry contains the media descriptor (`0xFFFB` probably), and the second entry stores the end-of-file marker (usually `0xFFFF`). The remaining entries in the FAT correspond to a data cluster in the data area, and can have the following values:

Value	Meaning
`0x0000`	Cluster is free
`0x0002–0xffef`	Cluster in use. Specifies the next cluster in the file
`0xfff0–0xfff6`	reserved (shouldn't be there)
`0xfff7`	Bad cluster
`0xfff8–0xffff`	Cluster in use. Last cluster in the file.

Since the first cluster in the data area is numbered 2, the value `0x0001` should not never occur.

Remember that all FAT entries refer to logical clusters. You will need to compute the sector number: **sector = (1 + FAT1Sectors + FAT2Sectors + RootSectors) + cluster_number - 2**.

3.3 Directories

Directories are really just files with a special format. Each 512-byte sector in a directory contains 16 directory entries (each of which is 32 bytes long). Each directory entry represents some file or subdirectory, within that directory. A directory entry consists of the following information.

Starts at (offset)	Length (in bytes)	Description
0	8	Filename
8	3	Extension
11	1	Attributes (see below)
12	2	Reserved
14	2	Creation Time
16	2	Creation Date
18	2	Last Access Date
20	2	Reserved (unused in FAT16)
22	2	Last Write Time
24	2	Last Write Date
26	2	First Logical Cluster
28	4	File Size (in bytes)

Note that if the first byte of the filename is 0xE5, then the directory entry has been deleted and is now available. If the first byte in the filename is 0x00, then the entry is free, and all remaining entries are also free.

For this project, we have been ordered by recently elected President M. Kondo to use **Standard Short Filenames**. As such, file (or directory) names and extensions are always 8 and 3 bytes respectively. They are also not null-terminated. If the file name is shorter than 8 characters, it is padded with spaces (0x20). This also applies for extensions. File (and directory) names should always be stored in uppercase in FAT16. Subdirectories will also contain two special entries named "." and "..", which link to the current directory and the parent directory respectively.

The Attributes field has 8 bits where each bit refers to a different attribute of the entry. The bits have the following meaning.

Bit	Attribute
0	Read Only
1	Hidden
2	System
3	Volume Label
4	Directory
5	Archive
6	—
7	—

Bit zero is the *least significant bit*, so if a directory entry has the Attribute `0x12`, then the entry is a hidden directory. If you see an attribute byte of 0x0F, you can safely ignore it. This is part of a long-file name (an extension that was added later), which is something you don't have to handle in this assignment. The autograder will not test this.

The **First Logical Cluster** tells you where the file's (or subdirectory's) content is stored. This just tells you where to start. If the file has more than 1 cluster, you will need to use the FAT to find the remaining clusters. If the First Logical Cluster is 0x000, then this refers to the root directory (remember that the first cluster in the data area is logical cluster 2). If the First Logical Cluster were `0x004`, then you know that the first cluster worth of data is stored in cluster #4. If you look up the entry 4 in the FAT, you might see `0xFFF` indicating that the file (or directory) only has one cluster. Or, you might see something like `0x005` indicating that the next cluster in the file is cluster #5. Each FAT entry points to the next cluster in that file or directory. To get the rest of the data, you continue iterating through the FAT, until you have found all clusters in the file.

4 Handling Errors

Your library needs to handle a variety of error conditions. The POSIX (2001, 2008) and C99 standards introduced a bunch of standard error cases (see the man page for `errno` for a list) and while you don't have to support all of them, I expect your library to handle the following errors in the following way (see the header file for more details).

The API functions follow two basic patterns. Some return an int "result code", which should be zero (0) for success, and some negative value for an error. I should be able to pass these result codes to your `cat16_strerror`

function to get the corresponding error message. The rest return a pointer to either a struct or some data read from a file. If an error occurs, these functions should return NULL, and (in all but `cat16_mount`) an error code should be stored in the `cat16_volume_t` struct. Calling `cat16_vstrerror` immediately after a failed call (that returns NULL) should return the error message for the last error.

Your code should specifically handle the following error cases:

- **ENOSPC** — No space left on device. You should set this error if you are unable to find space to write the requested file or create the new requested directory. This would also be appropriate if you run out of room in the root directory.
- **EINVAL** — Invalid argument. The test passed an invalid argument (maybe a NULL pointer) to the library.
- **ENAMETOOLONG** — Filename too long. Requested filename doesn't fit in the 8.3 limitations of the project.
- **ENOENT** — No such file or directory. The file you wanted to read, or the directory you want to list, or modify, does not exist.
- **ENOTDIR** — Not a directory. You tried to do something to a file that is only allowed for directories (like listdir or rmdir).
- **EISDIR** — Is a directory. You called `cat16_unlink` on a directory.
- **ENOTEMPTY** — Directory not empty. The `cat16_rmdir` function can only delete a directory once the files have been removed.

You should use the standard error messages for each case. You can use `strerror` to get the messages.

4.1 Testing your program

You should test your program on a variety of disk images. I will give you a few test images (including a blank image). You can and should also create your own. The `mtools`, a collection of tools for manipulating DOS disk images, are installed on the lab machines. The `man` page for `mtools` is a good place to start.

Listing 3 shows one of my scripts for creating a test image (**images/simple.img**) from a blank image (called **blankfloppy.img**) and some random files located in the `test_content` directory.

Listing 3: Making your own test images.

```
# create a blank disk image
dd if=/dev/zero of=$@ bs=1M count=4
mkfs.fat -F 16 -S 512 -s 1 blank.img

# customize it
cp blank.img tmp.img
mcopy -i tmp.img test_content/test.txt ::A.TXT
mcopy -i tmp.img test_content/deleted.txt ::B.TXT
mcopy -i tmp.img test_content/kitty.jpg ::CAT.JPG
mmd -i tmp.img ::IMGS
mcopy -i tmp.img test_content/kitty.jpg ::IMGS/KITTY.JPG
mdel -i tmp.img ::B.TXT
mv tmp.img images/simple.img
```

The `mtools` has one peculiarity you should be aware of. Depending on how they were compiled, you can run into strange behaviors related to long file names. If you use all uppercase filenames and directory names in your image, and make sure that all names are shorter than 8 characters (plus a 3 char extension) it should always use short filenames, which is all you are required to support.

You should also write your own test programs, like you did in the previous project. These programs should read files, write files, add and remove directories, etc. I will provide one test in the git repository. You might also find it helpful to use the `fsck.vfat` program to check your images for errors after your test run (to make sure you didn't break the image). For example, the following command will check a disk image and report many common errors, like whether your FATs don't match or you have broken FAT chains: `fsck.fvat -nv test.img`.

Your library should compile and run on the lab machines. Lack of thorough testing on the lab machines is likely to lead to suboptimal outcomes.

5 Submission Instructions

This project has two deadlines. Absolutely no late assignments will be accepted, and deadline extensions typically require a documented medical emergency or natural disaster.

Compile deadline: 4:00 PM, October 16th — This is a preliminary deliverable worth 10 points. For this, you simply need to submit a valid tgz file (more info below on this) with code and a Makefile that compiles without

any errors or warnings and produces a library, called `cat16.a`. This compiled library does not need to work correctly, but it does need to have all of the required functions, or some of my tests will not compile. For this deliverable, submit to the compile3 assignment on Canvas.

Functional deadline: 4:00 PM, November 10th — This is the final submission deadline worth the remaining 60 points (50 functional + 10 style). For this deliverable, submit to the project3 assignment on Canvas.

Your program must be written in C. When your project is complete, archive your source materials, using something like the following command:

```
> tar cvzf project3.tgz README Makefile <list of source files>
```

The **Makefile** should compile your program (by running **make**). Your program should compile without producing any errors or warnings.

The **README** file should include your name, a short description of your project, and any other comments you think are relevant to the grading. It should have two clearly labeled sections titled with KNOWN PROBLEMS, and DESIGN. The KNOWN PROBLEMS section should include a specific description of all known problems or program limitations. The DESIGN section should include a short description of how you designed your solution (especially anything you thought was interesting or clever about your solution). You should also include references to any materials that you referred to while working on the project. Please do not include special instructions for compilation. The compilation and running of the tests will be automated (see details below) and your instructions will not be followed.

Please make sure you include only the source files, not the object files. Before you submit this single file, please use the following command to check and make sure you have everything in the `project3.tgz` file, using the following script:

```
> tar xvzf project3.tgz
```

This command should put all of your source files in the current directory. It should not create any subdirectories. Running `make` after extracting the files should build your `cat16.a` library, which should end up in the current directory.

Submit your `project3.tgz` file via **Canvas**. You must name your archive **project3.tgz**, and your program must compile and work. We will compile your program using your **Makefile**, and it must be named "`cat16.a`".

6 Grading

Your project will be graded based on the results of functional tests the design of your code. We will run a variety of tests to make sure it works properly. Your program should not crash. Specifically, you will receive 10 points if your code successfully compiles, and 10 points for code style and readability. As always there will be a project quiz worth 30 points. The rest of your score (50 points) will be determined by your library's functionality.

Your source materials should be readable, properly documented and use good coding practices (avoid magic numbers, use appropriately-named variables, etc). Your code should be `-Wall` clean (you should not have any warnings during compilation). **Our testing of your code will be thorough. Be sure you test your application thoroughly.**

7 A building block for Project 4

I am giving you a little extra time on this project, because this project (and project 2) will be building blocks for project 4. In project 4, you will create a server, using sockets, that will allow clients to read from and manipulate FAT16 disk images remotely over the network. I'm giving you the heads up (and extra time) because doing a sloppy job on this project (or not completing it) could make the next project significantly more difficult.

8 Collaboration

You may work independently or with one other person on this project. You must *not* discuss the problem or the solution with any classmates who are not your partner. All of your code must be your own code. **If you work with a partner, only one of you should submit and both of your names should be listed clearly at the top of your README file.**

You may, of course, discuss the project with me, and you may discuss conceptual issues related to the project that we have already discussed in lecture, via Piazza (do not post any code or implementation details of the algorithms). **Collaborating with any peers (who are not your partner) or a generative AI on this project, in any other manner will be treated as academic misconduct.**